

cindex 索引处理程序

Wenjian Chern*

2026 年 9 月 22 日

目录

§ 1	简介	1	3.3	实用类型	20
1.1	选项说明	1	3.4	PCRE2 正则表达式	23
1.2	索引项输入语法	2	§ 4	可加载的外部库	27
§ 2	格式文件	3	4.1	cindex.table	27
2.1	ist 格式文件	3	4.2	icu.table	28
2.2	使用 lua 定制	6	4.3	digest 哈希函数	28
§ 3	lua 类型	7	4.4	LPeg	29
3.1	字符相关	7	4.5	argparse	29
3.2	索引条目	17	§ 5	代码索引	30

§ 1 简介

cindex 是通用的索引制作工具，它的作用类似于 makeindex 和 zhmakeindex，功能和用法也和它们相似。但支持使用 lua (luajit 2.1) 进行特殊定制。

输入输出的格式可以由 ist 格式确定，但也支持使用 lua 文件文件进行定制。

与 makeindex 一样，cindex 默认的输入/输出格式为 idx/ind。此外，cindex 还支持一种特殊的输入格式 ikv，使用这个格式无需对索引条目进行复杂的转义。同时 ikv 完全兼容 idx，也就是说，一个合法的 idx 文件就是一个合法的 ikv 文件。

如果没有显式指定，第一个输入文件的主文件名将用于，确定输出文件和日志文件的主文件名，它们并不继承该输入文件的目录。若未找到某输入文件，且该文件没有扩展名，则会加上 .idx 再次寻找。目前，cindex 使用 kpsewhich 命令行工具查找相关文件。

不像 zhmakeindex，在没有提供格式文件的情况下，cindex 并不寻找默认的格式文件，而是直接使用程序的默认设置。

cindex 支持多级索引，默认的实现支持至多 5 级，与之相比，makeindex/zhmakeindex 只支持最多 3 级。若使用 lua 定制，则层级没有限制，由具体的格式决定。

cindex 只接受 UTF-8 编码的文件，不论是格式文件，亦或是输入输出文件，必须使用 UTF-8 编码。

1.1 选项说明

```
cindex [-c] [-i] [-o <ind>] [-q] [-r] [-s <style>] [-t <log>] [--strict]
      [-z <sorts>] [--kv] [{<index0> <index1> ...} [-- <extras>]
```

*E-mail: longaster@163.com

`cindex [-h, --help]`

`cindex [--license]`

`cindex query [-q] <kind> <text>`

`cindex query [-h, --help]`

-c 缩索引项排序项前后的空格。默认情况下，排序项中的空格会被保留。此选项对 `ikv` 格式的索引无效，默认情况下 `ikv` 格式总是忽略行首空格，保留行末空格。

-i 从标准输入流（`stdin`）读入索引项。如果使用了该选项，并且没有使用 `-o` 选项，则排序后的索引将输出到标准输出流（`stdout`）。

-i <ind> 设置输出索引文件为 `<ind>`。如果没有指定该选项，默认的输出文件名是第一个输入文件 `<index0>` 的主文件名加上 `.ind` 扩展名。

-q 静默模式，不向标准错误流（`stderr`）显示信息。默认情况下处理过程与错误信息会同时在 `stderr` 与日志文件中输出。

-r 禁止隐式页码区间构造，要求页码区间必须使用显式区间符号生成。默认情况下，三个或三个以上连续的页码会自动合并为一个页码区间（如 `1—5`）。

-s <style> 设置 `<style>` 为格式文件。没有默认值。如果没有扩展名，会使用 `.ist` 作为扩展名。`cindex` 会首先在当前目录查找格式文件，如果找不到则调用调用 `kpsewhich` 命令行工具查找，可使用 `KPSEWHICH_EXE_FILE` 环境变量设置 `kpsewhich` 二进制文件的位置。

-t <log> 设置 `<log>` 为日志文件。默认情况下，会使用第一个输入文件 `<index0>` 的主文件名加上 `.ilg` 扩展名作为日志文件。

--strict 严格区分不同嵌入命令的页码。默认情况下，在页码区间处理时，会将如果页码左区间的嵌入命令与右区间不匹配，会以左区间为准；而如果使用该选项，则要求左右区间的命令类型必须严格匹配。

-z <sorts> 设置分组与排序方式为 `<sorts>`。由 `cindex` 直接支持的排序方式包括 `pinyin/reading`, `bihua/stroke`, `bushou/radical`。默认值为 `pinyin`，即中文按拼音分组排序。

`<sorts>` 为以西文逗号分隔的列表。默认情况下，`cindex` 只检查 `<sorts>` 的第一项，可使用 `lua` 定制。

--kv 标记输入文件均为 `ikv` 格式。`cindex` 会检查输入文件的扩展名，如果扩展名为 `ikv`，则将其视为 `ikv` 格式。

-- <extras> 输入额外的选项。可由 `lua` 进一步处理。

如

```
cindex -s process.lua index.idx -- some other options
```

则 `<extras>` 为表 `{ "some", "other", "options" }`。

1.2 索引项输入语法

`idx` 格式的输入文件，其语法和 `makeindex/zmakeindex` 别无二致。该格式下，每行一个索引项，允许空行，但不得跨行。

如下是几个索引项的例子：

```
\indexentry{姓名}{15}
```

```
\indexentry{ExplSyntaxOn={\verbatim@font !\verb*&!\ExplSyntaxOn&}|hdpindex{usage}}{MMMMI-6}
```

ikv 格式的输入文件，每个索引项分为“头”和“具体层级”两个部分：

```

<页码><分隔符><Range><分隔符><命令><分隔符><层级数>
  <排序键1>
  <显示文本1>
  <排序键2>
  <显示文本2>
  ...

```

比如假定使用默认的分隔符 `,`，上述 idx 格式的输入可改写为

```

15,,1
  姓名
  姓名
MMMMI-6,,hdindex{usage},1
  ExplSyntaxOn
  \verbatim@font \verb*&\ExplSyntaxOn&

```

值得注意的是，分隔符前后不能有多余的空格（分隔符本身可以包含空格，可以是多个字符），即便排序键和显示文本相同，也不能省略其中任何一个。排序键行和显示文本行行首的空格可以省略，`cindex` 默认会忽略行首空格。

ikv 格式完全兼容 idx 格式，只把扩展名改为 ikv，而内容本身不做改变，同样也能被 `cindex` 正确识别。比起 idx 格式，ikv 无需（也不能）做任何转义。在 ikv 中使用 idx 格式的索引项则除外。

在 $\text{\LaTeX}$ 中使用时，索引项输入文件一般是在 $\text{\LaTeX}$ 编译过程中自动生成的。

§ 2 格式文件

`cindex` 除了支持 `makeindex` 的 `ist` 格式文件外，还支持使用 `lua` 定制。

2.1 ist 格式文件

`cindex` 支持的 `ist` 格式文件与标准 `makeindex` 语法完全相同，内容也基本上相同，同时增加了少量控制中文分组输出的格式，它们大多来源于 `zhmakeindex`。可以在 `cindex` 中使用标准 `makeindex` 的格式文件。

格式文件指明了 `.idx` 输入文件和最终输入文件的格式。该文件在当前工作目录或在 `TEXMF` 树中由 `kpathsea` 库查找。一个格式文件由一组 <关键字> <属性> 的列表组成。<关键字> 分为输出和输出两类。<关键字> <属性> 对可以以任何顺序出现。以字符 `%` 开头的行是注释，<属性> 可以有不同的类型，字符串是以任意双引号 `"` 或反引号 ``` 界定的串，字符是以单引号 `'` 界定的单个字符，数字是一个整数。其中，双引号和单引号界定的串和字符，可以使用反斜线 `\` 符号作为转义符，以得到特殊字符或引号本身；而反引号界定的串是表示原始字符串，它不使用转义符。格式文件中没有指定的项目会使用其默认值。

表 1: 与 `makeindex` 兼容的输入格式

关键字	类型	默认值	意义
<code>keyword</code>	字符串	<code>"\indexentry"</code>	索引关键字
<code>arg_open</code>	字符	<code>'{'</code>	参数的开定界符

关键字	类型	默认值	意义
arg_close	字符	'}'	参数的闭定界符
range_open	字符	'('	页码范围的开定界符
range_close	字符	')'	页码范围的闭定界符
level	字符	'!'	索引项层次分隔符
actual	字符	'@'	(区别于排序项的) 实际输出项标志符
encap	字符	' '	页码和特殊命令指示符
quote	字符	'\"'	引号 (转义符)
escape	字符	'\\'	转义 quote 的符号, 在它后面的引号不作转义符解释
page_compositor	字符串	"-"	复合页码分隔符

表 2: cindex 特有的输入格式

关键字	类型	默认值	意义
comment	字符	'%'	ist 格式行注释的开始符
separator	字符串	", "	ikv 格式“头”行的分隔符

表 3: 与 makeindex 兼容的输出格式

关键字	类型	默认值	意义
preamble	字符串	"\\begin{theindex}\n"	索引引导言代码
postamble	字符串	"\n\n\\end{theindex}\n"	索引末尾代码
group_skip	字符串	"\n\n \\indexspace\n"	组间垂直间距
headings_flag	数字	0	控制显示分组名标题的旗标
heading_prefix	字符串	""	分组名标题的前缀
heading_suffix	字符串	""	分组名标题的后缀
symhead_positive	字符串	"Symbols"	当旗标 headings_flag 为正数时, 符号的标题
symhead_negative	字符串	"symbols"	当旗标 headings_flag 为负数时, 符号的标题
numhead_positive	字符串	"Numbers"	当旗标 headings_flag 为正数时, 数字的标题
numhead_negative	字符串	"numbers"	当旗标 headings_flag 为负数时, 数字的标题
item_0	字符串	"\n \\item "	第 0 级条目间分隔
item_1	字符串	"\n \\subitem "	第 1 级条目间分隔
item_2	字符串	"\n \\subsubitem "	第 2 级条目间分隔
item_01	字符串	"\n \\subitem "	第 0/1 级条目间分隔
item_x1	字符串	"\n \\subitem "	第 0/1 级条目间分隔, 其中第 0 级无页码
item_12	字符串	"\n \\subsubitem "	第 1/2 级条目间分隔
item_x2	字符串	"\n \\subsubitem "	第 1/2 级条目间分隔, 其中第 1 级无页码

关键字	类型	默认值	意义
delim_0	字符串	", "	第 0 级条目与页码分隔
delim_1	字符串	", "	第 1 级条目与页码分隔
delim_2	字符串	", "	第 2 级条目与页码分隔
delim_n	字符串	", "	多个页码的分隔
delim_r	字符串	"--"	页码范围符号
delim_t	字符串	""	将插入到在页码列表末尾。如果页码列表为空，此项无效果。
encap_prefix	字符串	"\""	页码特殊指令前缀
encap_infix	字符串	"{"	页码特殊指令中缀
encap_suffix	字符串	"}"	页码特殊指令后缀
page_precedence	字符串	"rnaRA"	不同类型页码的次序，默认值表示小写罗马、阿拉伯数字、小写字母、大写罗马、大写字母
suffix_2p	字符串	""	在 2 页的页码范围中代替 delim_r 和第二个页码
suffix_3p	字符串	""	在 3 页的页码范围中代替 delim_r 和后面的页码
suffix_mp	字符串	""	在更多页的页码范围中代替 delim_r 和后面的页码

表 4: cindex 特有的输出格式

关键字	类型	默认值	意义
stroke_prefix	字符串	""	笔画数前缀
stroke_suffix	字符串	" 画"	笔画数后缀
radical_prefix	字符串	""	部首前缀
radical_suffix	字符串	"部"	部首后缀
radical_simplified_flag	数字	1	是否输出简化部首的标志
radical_simplified_prefix	字符串	" ("	简化部首前缀
radical_simplified_separator	字符串	"、"	多个简化部首间的分隔符
radical_simplified_suffix	字符串	")"	简化部首后缀

表 5: 未实现的输出格式

关键字	类型	默认值	意义
setpage_prefix	字符串	"\n \\setcounter{page}{"	页码设置前缀
setpage_suffix	字符串	"}\n"	页码设置后缀
line_max	数字	72	最大行长度，超出长度会自动折行
indent_space	字符串	"\t\t"	自动折行的缩进
indent_length	数字	16	indent_space 的长度

2.2 使用 lua 定制

使用 lua 文件（扩展名为 lua 或 cilua）作为格式文件时，该 lua 文件必须返回 nil 或表。cindex 会在合适的时候查询这张表的字段。如果相应字段不存在，cindex 会使用默认值。

```
parse_indices_string: function(  
  file_path: string|nil,  
  lines: StrRef,  
  iv_ikv: boolean  
) → function() → IndexEntryRefMut | table | nil | false;
```

parse_indices_string

解析传入文件的内容。此函数应当返回一个迭代器函数，每次调用这个迭代器，都会把它的返回值作为一个结构化的索引条目，直到它返回 nil，如果返回 false，相当于忽略该条目。如果函数返回的是表，其结构见 `IndexEntryRef.as_table`，否则，应当返回 `IndexEntryRefMut`。

```
parse_index_line: function(line: StrRef)  
  → IndexEntryRefMut | table | false;
```

parse_index_line

此函数应当解析某个输入行，返回一个结构化的索引条目，如果返回 false，相当于忽略该条目。如果函数返回的是表，其结构见 `IndexEntryRef.as_table`，否则，应当返回 `IndexEntryRefMut`。

当它与 `parse_indices_string` 同时存在时，cindex 只使用 `parse_indices_string`，因为它覆盖更为广泛。

```
empty_indices_callout: function(array: IndexEntryArray);
```

empty_indices_callout

当初次解析完后，如果没有任何有效的索引项，则调用此函数。

```
page_precedence: function | { string, .. } | string;
```

page_precedence

不同种页码的次序。

页码的种类为 "Arabic"(n) | "roman"(r) | "Roman"(R) | "alpha"(a) | "Alpha"(A) | "ZhDigits" | "ZhNumber"，另见 `IndexEntryRef.as_table`。如果是字符串而非以表给出，则字符串的每一项应为括号内的字符。如 { "roman", "Arabic", "alpha", "Roman", "Alpha", "ZhDigits", "ZhNumber" } 和 "rnaRA" 都表示相同的优先级，这也是默认的优先级。如果是函数，则其返回值必须为表或字符串或 nil。

```
group_detect: function(entry: MergedEntryRef) → string;
```

group_detect

判断并返回条目的组别。

cindex 预定义的组别及其排序为 nil, Symbols, Numbers, A ~ Z, BiHua1 ~ BiHua64（笔画数 1 至 64，超过 64 画的按 64 画计），BuShou1 ~ BuShou214（214 个康熙部首）。

```
group_compare: function(
  lhs: string,
  rhs: string,
  lhs_counts: number,
  rhs_counts: number,
) → boolean;
```

group_compare

组别的排序。如果 lhs 要排在 rhs 前面，则返回 true。目前 cindex 使用的是 lua 内置的 table.sort 函数，它是不稳定排序。lhs_counts 和 rhs_counts 分别是对应组别在合并后所拥有的条目数。

```
entry_compare: function(
  lhs: MergedEntryRef,
  rhs: MergedEntryRef,
  group: string,
) → boolean;
```

entry_compare

条目的排序。如果 lhs 要排在 rhs 前面，则返回 true。目前 cindex 使用的是 lua 内置的 table.sort 函数，它是不稳定排序。group 为条目所属的组别。

如果不同组别的排序方式不同，也可使用特化的 entry_compare_⟨group⟩ 函数，比如 entry_compare_Numbers 如果存在，则对于 Numbers 这个组别，在对其条目排序时，会直接使用这个函数作为比较函数。对于按笔画数和部首的组别，除了会检查 entry_compare_BiHua2 这种外，cindex 还检查不带数字的排序函数：entry_compare_BiHua。cindex 会优先使用更加具体的比较函数。

```
output_style: { .. }
```

output_style

当使用 lua 作为格式文件时，便无法同时使用 ist 格式了。为此，可使用 output_style 表作为替代。这张表的字段就是 ist 中的相应关键字。

可用的关键字见表 3 和表 4。

§ 3 lua 类型

3.1 字符相关

一般情况下，cindex 中保存数据的类型都能使用 == nil 来判断它是否为无效值。

```
Char(number) → Char;
Char(Char) → Char;
Char(string) → Char | nil;
```

Char

该类型的构造器。返回一个 Char，这是 UTF-32 编码的 Unicode 字符。返回值可能并非有效的 Unicode 字符。它与 nil 可进行相等性比较，如果为无效值，则它与 nil 判定为相等（但不可替代 nil）。

如果输入为字符串，则可能返回 nil。如果字符串的第一个字符为有效的 UTF-8 编码的字符，就返回这个字符。

Char 实现的元方法。

Char.__eq

Char.__lt

Char.__le

Char.__tostring

```
Char.is_valid(self) → boolean;
```

Char.is_valid

判断是否为有效的 Unicode 字符。

```
local c = Char('汉');
print(c.is_valid());
local c = Char(0xDFFF);
print(c.is_valid(), c == nil);
.....
true
false true
```

```
Char.from_number(number) → Char;
Char.to_number(self) → number;
```

Char.from_number
Char.to_number

从数字转为 Char 或转为数字。

```
Char.cmp(Char?, Char?) → number | nil;
```

Char.cmp

比较两个字符，如果前者小于后者，返回负数，如果大于后者，返回正数，如果相等，返回 0。如果二者之一为 nil，若是字符无效，返回 0，否则返回 nil。

```
Char.from_string(string) → (Char | nil, boolean);
```

Char.from_string

若字符串开头为无效的 UTF-8 序列，则返回 nil，此时第二个返回值总为 false。当输入为有效的 UTF-8 字符串时，第一个返回值总是一个有效的 Char，当字符串只有一个字符时，第二个返回值为 true。

```
print(Char.from_string('汉'));
print(Char.from_string('汉字'));
print(Char.from_string('\xFF'));
.....
汉 true
汉 false
nil false
```

```
Char.chars(string) → function() → Char | nil;
```

Char.chars

返回一个迭代器函数，这个函数依次返回 UTF-8 字符串的所有字符。

```
for c in Char.chars('CJK汉字') do
    print(c);
end;
.....
C
J
K
汉
字
```

```
Char.has_han_info(self) → boolean;  
Char.han_info(self) → CjkInfo | nil;
```

Char.has_han_info
Char.han_info

判断该字符是否有 CJK 相关的信息，或获取该信息。

Unicode 区块 CJK Unified Ideographs、CJK Extension A ~ J、CJK Compatibility Ideographs、CJK Compatibility Ideographs Supplement 中的所有字符必定有此信息，但“〇”（U+3007）不再此列。

```
print(Char('L'):has_han_info());  
print(Char('汉'):has_han_info());  
print(Char('〇'):has_han_info()); -- U+3007  
.....  
false  
true  
false
```

```
Char.han_ordered_strokes(self) → OrderedStrokes | nil;
```

Char.han_ordered_strokes

返回汉字的笔顺。字形为中国大陆字形。

```
print(Char('L'):han_ordered_strokes());  
print(Char('汉'):han_ordered_strokes());  
print(Char('〇'):han_ordered_strokes()); -- U + 3007  
.....  
nil  
44154  
nil
```

```
Char.script(self) → number;
```

Char.script

返回字符所属的 Unicode Script 值，number 为内部句柄，可使用 `icu.table.Script` 表查询该值代表的字符串。

```
local icu_data = require("icu.table");  
print(icu_data.Script[ Char('L'):script() ]);  
print(icu_data.Script[ Char('汉'):script() ]);  
print(icu_data.Script[ Char('〇'):script() ]);  
.....  
Latin  
Han  
Han
```

```
Char.general_category(self) → number;
```

Char.general_category

返回字符所属的 Unicode General Category 值，number 为内部句柄，可使用 `icu.table.GeneralCategory` 表查询该值代表的字符串。

```
Char.is_alphabetic(self) → boolean;
```

Char.is_alphabetic
Char.is_numeric
Char.is_math

判断字符是否为字母/数字/数学符号，范围涵盖所有 Unicode 字符。

```
Char.is_ideographic(self) → boolean;
```

Char.is_ideographic

判断字符是否具有 Unicode Ideographic 属性。不单单只有 CJK 字符有此属性。

```
Char.simple_fold(self) → Char;
```

```
Char.simple_fold
```

折叠字符的大小写。字符在折叠大小写之后可进行忽略大小写的比较。

若要对字符串进行忽略大小写的比较，可使用 `StrRef.eq_ignore_case`。

不可变的 UTF-8 编码的字符串。它并不持有这字符串。

```
StrRef
```

`StrRef` 实现的元方法。它除了可以与另一个 `StrRef` 进行比较外，还能与 `string` 进行比较。当它与 `nil` 比较相等时，仅当它指向空指针时，被判定为与 `nil` 相等。

```
StrRef.__eq  
StrRef.__lt  
StrRef.__le  
StrRef.__len  
StrRef.__tostring
```

```
StrRef.is_nil(self) → boolean;
```

```
StrRef.is_nil  
StrRef.is_empty
```

`StrRef.is_nil` 判断它是否指向空指针，`StrRef.is_empty` 当 `StrRef.is_nil` 为真或其长度为 0 时返回 `true`。

```
StrRef.from_ptr(const StrRef*) → StrRef | nil;  
StrRef.unsafe_new(StrRef | string) → StrRef;  
StrRef.unsafe_new(const uint8_t*, size_t?) → StrRef;
```

```
StrRef.from_ptr  
StrRef.unsafe_new
```

不安全的构建方法。当它指向的值被销毁后，它本身不一定被销毁，因此可能出现悬垂引用。

```
StrRef.to_string_escaped(self, all: boolean) → string;
```

```
StrRef.to_string_escaped
```

对部分不可见字符转义后输出。如果第二个参数被视为真，则对所有字符都进行转义。每个字符的转义形式为 `\u{HHHH..}`，使用大写的 16 进制。

```
print(StrRef.unsafe_new('\x1F汉OL'):to_string_escaped());  
print(StrRef.unsafe_new('\x1F汉OL'):to_string_escaped(true));  
.....  
\u{001F}汉OL  
\u{001F}\u{6C49}\u{3007}\u{004C}
```

```
StrRef.cmp(StrRef, StrRef) → number;  
StrRef.cmp(StrRef, string) → number;  
StrRef.cmp(string, StrRef) → number;
```

```
StrRef.cmp
```

比较两个字符串。若前者小于后者，返回负数，若等于，返回 0，若大于，返回正数。

```
StrRef.eq_ignore_case(StrRef | string, StrRef | string) → boolean;  
StrRef.cmp_ignore_case(StrRef | string, StrRef | string) → number;
```

```
StrRef.eq_ignore_case  
StrRef.cmp_ignore_case
```

忽略大小写进行比较。若前者小于后者，返回负数，若等于，返回 0，若大于，返回正数。

某些字符在折叠大小写之后可能是多个字符，因此依次对 `Char.simple_fold` 的结果进行比较，其结果并不一定与 `StrRef.eq_ignore_case` 返回的值相等。

```
print(StrRef.eq_ignore_case('Ab汉', 'ab汉'));  
print(StrRef.eq_ignore_case('aß', 'ASS'));
```

```
print(StrRef.eq_ignore_case('aß', 'AS'));
```

```
.....
true
true
false
```

```
StrRef.slice(self, start: number, end_: number?) → StrRef;
```

StrRef.slice

返回从 **start** 到 **end_**（不含）的部分，索引从 **0** 开始。如果 **end_** 未给出或超过字符串的长度，则该值视为字符串的长度。索引以字节为准，而不是字符。

一般来讲，凡是 **cindex** 的类型，它们的方法使用到索引时，都是从 **0** 开始，若是需要数字的范围，则都是前闭后开的。

```
print(StrRef.unsafe_new('CJK汉字'):slice(1, 6));
```

```
.....
JK汉
```

```
StrRef.head(StrRef | string) → number | nil;
```

StrRef.head

返回字符串的第一个字符。

```
print(StrRef.unsafe_new('汉字'):head());
```

```
.....
27721
```

```
StrRef.chars(self) → function() → number | nil;
```

StrRef.chars

返回一个迭代器函数，这个函数依次返回 UTF-8 字符串的所有字符。

```
StrRef.lines(self) → function() → StrRef | nil;
```

StrRef.lines

返回一个迭代器函数，这个函数依次返回 UTF-8 字符串的所有行。行结束符不会包含在结果字符串中。

```
local s = "abc\n汉字\n什么";
for line in StrRef.unsafe_new(s):lines() do
  print(line);
end
```

```
.....
abc
汉字
什么
```

```
StrRef.split(self, pattern: nil | Char | StrRef | string)
→ function() → StrRef | nil;
```

StrRef.split

返回一个迭代器函数，这个函数依次返回 UTF-8 字符串的子串。分割方式由 **pattern** 指定。

如果模式为 **nil**，则按连续的 ASCII 空白分割，此时，它总不返回空串；否则按指定的字符或字符串分割。如果字符串以 **pattern** 结尾，那么最后一项就是这个 **pattern** 之前的部分子串。

如果输入为空串，则返回的第一个值就是 **nil**。

```
local s = "::std::os::ffi::";
for part in StrRef.unsafe_new(s):split("::") do
    print(part);
end
```

```
std
os
ffi
```

```
local s = " std os ffi \n ";
for part in StrRef.unsafe_new(s):split() do
    print(part);
end
```

```
std
os
s
ffi
```

```
StrRef.graphemes(self) → function() → StrRef | nil;
```

StrRef.graphemes

返回一个迭代器函数，这个函数依次返回每个 **Grapheme Cluster**。

```
local s = "🐼AB汉字。";
for cluster in StrRef.unsafe_new(s):graphemes() do
    print(cluster, #cluster);
end
```

```
🐼 4
A 35
A 1
B 1
汉 3
字 3
。 3
```

```
StrRef.unicode_words(self) → function() → StrRef | nil;
```

StrRef.unicode_words

返回一个迭代器函数，这个函数依次返回每个单词。

```
StrRef.split_word_bounds(self) → function() → StrRef | nil;
```

StrRef.split_word_bounds

返回一个迭代器函数，这个函数返回一个根据 **Word Boundaries** 分隔开来的子串。

```
local s = [[ABC: ( “汉🐼字” )。]];
for word in StrRef.unsafe_new(s):unicode_words() do
    print(word);
end
```

```
ABC
汉
```

字

```
local s = [[ABC: ( “汉👉字” )。]];
for part in StrRef.unsafe_new(s):split_word_bounds() do
    print(part);
end
```

```
.....
ABC
:

(
“
汉
👉
字
”
)
。
```

```
StrRef.trim_ascii_spaces() → StrRef | nil;
StrRef.trim_ascii_spaces(start: boolean) → StrRef | nil;
StrRef.trim_ascii_spaces(start: boolean, end: boolean) → StrRef | nil;
```

StrRef.trim_ascii_spaces

返回删除字符串前后空格的结果。如果是有效的字符串，不会返回 `nil`。默认情况下，前后的空格都被删除。

```
StrRef.is_emoji(self) → boolean;
```

StrRef.is_emoji

判断字符串是否为 Emoji 序列，有的 Emoji 是由多个 Unicode 字符组成。

```
print("#😄", StrRef.unsafe_new("😄"):is_emoji());
print("#👉", StrRef.unsafe_new("👉"):is_emoji());
print("#👉", StrRef.unsafe_new("👉"):is_emoji());
print("#👉", StrRef.unsafe_new("👉"):is_emoji());
print("#汉", StrRef.unsafe_new("汉"):is_emoji());
print("#L", StrRef.unsafe_new("L"):is_emoji());
```

```
.....
4 true
8 true
17 true
35 true
3 false
1 false
```

保存 CJK 相关的信息。这个类型拥有其数据。一般通过 `Char.han_info` 获取。

CjkInfo

实现的元方法。

CjkInfo.__eq
CjkInfo.__tostring

```
CjkInfo.radical_from_kind(kind: StrRef | string) → number | nil;
```

CjkInfo.radical_from_kind

从康熙部首的种类返回其康熙部首的内部表示。种类是 [Unicode CJKRadicals.txt 数据](#) 的第一列，一般是 1 ~ 214 的数字，有的还可带 1 至 3 个西文单引号，如 123, 213'' 分别表示第 123 个康熙部首，以及第 213 个康熙部首的第 2 个变体。

```
print(CjkInfo.radical_from_kind("13"));
print(CjkInfo.radical_from_kind("213''"));
.....
12
244
```

```
CjkInfo.is_nil(self) → boolean;
```

CjkInfo.is_nil

判断是否为无效值。

```
CjkInfo.contains_char(Into<Char>) → boolean;
```

CjkInfo.contains_char

判断字符是否有 CJK 信息。它与 [Char.has_han_info](#) 完全一致。

```
print(CjkInfo.contains_char('L'));
print(CjkInfo.contains_char('汉'));
print(CjkInfo.contains_char(0x4E00));
print(CjkInfo.contains_char(0xDFFF));
.....
false
true
true
false
```

```
CjkInfo.kx_radical_char(self) → Char;
```

CjkInfo.kx_radical_char

返回康熙部首的具体字符，这字符位于 Unicode 的康熙部首相关区块。某些康熙部首的变体在康熙部首相关区块中没有对应值，因此，返回的字符可能无效。

```
CjkInfo.kx_ideography_char(self) → Char;
```

CjkInfo.kx_ideography_char

返回康熙部首的具体字符，这字符位于 Unicode 的 CJK 表意字相关区块。所有康熙部首及其变体在表意字区块都有对应值，因此有效的 [CjkInfo](#) 总返回有效的字符。

```
print(Char('汉'):han_info():kx_ideography_char());
print(Char('龙'):han_info():kx_ideography_char());
.....
水
龙
```

```
CjkInfo.kangxi_radical_number(self) → number | nil;
```

CjkInfo.kangxi_radical_number

返回康熙部首的号码，目前的返回值为 1 ~ 214。

```
print(Char('汉'):han_info():kangxi_radical_number());
print(Char('龙'):han_info():kangxi_radical_number());
```

```
.....
85
212
```

```
CjkInfo.kangxi_radical_ideography_slot(self)
→ number | nil;
```

CjkInfo.kangxi_radical_ideography_slot

返回康熙部首在 CJK 表意字区块中的对应值的代码点。康熙部首的变体被映射为原始的 214 个部首。

```
CjkInfo.kx_radical_kind(self) → StrRef | nil;
```

CjkInfo.kx_radical_kind

返回康熙部首的字符串表示。由数字和末尾的 0~3 个 ' 组成。

```
print(Char('汉'):han_info():kx_radical_kind());
print(Char('龙'):han_info():kx_radical_kind());
.....
85
212'
```

```
CjkInfo.ucd_additional_strokes(self) → number;
```

CjkInfo.ucd_additional_strokes

返回 Unicode 数据中该字符除部首外的笔画数，该值可能为负数。

```
CjkInfo.ucd_total_strokes(self) → number;
```

CjkInfo.ucd_total_strokes

返回 Unicode 数据中该字符的总笔画数。这与所选字形无关，另见 CjkInfo.total_strokes。

```
CjkInfo.total_strokes(self) → number;
```

CjkInfo.total_strokes

返回字符的总笔画数。不同字形标准，总笔画数可能不同，该函数使用中国大陆字形标准。它的值不一定和 CjkInfo.ucd_total_strokes 返回的一致。

```
CjkInfo.original_radical(self) → number | nil;
```

CjkInfo.original_radical

如果是康熙部首的变体，则返回其原始的康熙部首的内部表示，若就是原始部首，则返回原始部首的内部表示。对于有效的内部值，总是不返回 nil。

```
CjkInfo.mandarin(self) → nil | (StrRef, number);
```

CjkInfo.mandarin

如果该字符有拼音信息，第一个返回值是它最常见的拼音，第二个返回值是它的声调。

```
print(Char('汉'):han_info():mandarin());
print(Char('长'):han_info():mandarin());
.....
han 4
zhang 3
```

```
CjkInfo.cantonese(self) → nil | (StrRef, number);
```

CjkInfo.cantonese

如果该字符有粤语拼音信息，第一个返回值是它最常见的拼音，第二个返回值是它的声调。

```
Emoji(string | StrRef) → Emoji | nil;
```

Emoji

如果字符串是一个有效的 Emoji 序列，则返回这个 Emoji，Emoji 类型是 Owned。

把 emoji 转为字符串。

Emoji.__tostring

```
Emoji.name(self) → StrRef | nil;
```

Emoji.name

返回 emoji 的名称。一般情况下该方法不会返回 nil。

```
print(Emoji('😭'), Emoji('😭'):name());
print(Emoji('🇨🇳'), Emoji('🇨🇳'):name());
print(Emoji('👩'), Emoji('👩'):name());
print(Emoji('💋'), Emoji('💋'):name());
```

```
.....
😭 face with tears of joy
🇨🇳 flag: China
👩 woman facepalming: light skin tone
💋 kiss: woman, man, light skin tone
```

```
Emoji.group(self) → number;
```

Emoji.group

返回 emoji 组别的内部表示。可使用 `icu.table.EmojiGroup` 获取组别的名字。

```
local icu_data = require("icu.table").EmojiGroup;
print(Emoji('😭'), icu_data[Emoji('😭'):group()]);
print(Emoji('🇨🇳'), icu_data[Emoji('🇨🇳'):group()]);
print(Emoji('👩'), icu_data[Emoji('👩'):group()]);
print(Emoji('💋'), icu_data[Emoji('💋'):group()]);
```

```
.....
😭 Smileys And Emotion
🇨🇳 Flags
👩 People And Body
💋 People And Body
```

汉字的笔画。

OrderedStrokes

OrderedStrokes 实现的元方法。__ipairs 返回的索引号从 1 开始。

OrderedStrokes.__len
OrderedStrokes.__eq
OrderedStrokes.__lt
OrderedStrokes.__gt
OrderedStrokes.__ipairs

转为字符串的形式。字符串的每一项都是 1~5 的某个数字，按顺序分别代表：横、竖、撇、点（捺）、折（弯钩）。

OrderedStrokes.__tostring

```
OrderedStrokes[number] → number;  
OrderedStrokes.get(n: number) → number;
```

OrderedStrokes.__index
OrderedStrokes.get

获取第 n 画。 n 从 0 开始。数字为 1~5 的某个数字，按顺序分别代表：横、竖、撇、点（捺）、折（弯钩）。

3.2 索引条目

未合并的索引。

IndexEntryRef

IndexEntryRef 可以与 **nil** 进行比较，当其为无效值时，被视为与 **nil** 相等。

IndexEntryRef.__eq

```
IndexEntryRef.is_nil(self) → boolean;
```

IndexEntryRef.is_nil

判断是否为 **nil**。

```
IndexEntryRef.to_handle(self) → number;
```

IndexEntryRef.to_handle

返回一个句柄。一般来讲，**IndexEntryRef** 是不能作为数字的索引的，可转为句柄。

```
IndexEntryRef.levels_count(self) → number;
```

IndexEntryRef.levels_count

返回此条目的层级数。

```
IndexEntryRef.key_n(self, n: number) → StrRef | nil;  
IndexEntryRef.keys(self) → function() → StrRef | nil;
```

IndexEntryRef.key_n

IndexEntryRef.keys

返回该条目第 n 个排序关键字， n 从 0 开始。或返回一个迭代器函数。

```
IndexEntryRef.as_table(self) → table;  
table := {  
  levels: { { key?: string, display: string }, .. },  
  range?: "None" | "Open" | "Close",  
  page_commands?: string,  
  pages?: { { kind: string, value: number }, .. },  
  pages_raw?: string,  
}
```

IndexEntryRef.as_table

把条目表示为 **lua** 表。当前 **kind** 的值有：

- **Arabic**（阿拉伯数字），
- **alpha**（小写拉丁字母组成的 26 进制数字），
- **Alpha**（大写拉丁字母组成的 26 进制数字），
- **roman**（小写罗马数字），
- **Roman**（大写罗马数字），
- **ZhDigits**（中文数串，由“零〇一二三四五六七八九”组成），
- **ZhNumber**（中文数字）。

```
IndexEntryRef.is_same(lhs, rhs) → boolean;
```

IndexEntryRef.is_same

判断两个条目的排序关键字是否完全相等。

存储索引条目的数组。

IndexEntryArray

IndexEntryArray 实现的元方法。**__ipairs** 返回的索引数字从 1 开始。

IndexEntryArray.__len

IndexEntryArray.__ipairs

```
IndexEntryArray.is_nil(self) → boolean;  
IndexEntryArray.is_empty(self) → boolean;
```

IndexEntryArray.is_nil

IndexEntryArray.is_empty

判断数组是否为无效值，或其不含任何值。

```
IndexEntryArray[number] → IndexEntryRef;
IndexEntryArray.get(self, index: number) → IndexEntryRef;
```

```
IndexEntryArray.__index
IndexEntryArray.get
```

用数字索引数组，索引从0开始。若索引大于或等于数组的长度，**IndexEntryRef**并非一个有效值，可使用 `== nil` 或 `IndexEntryRef.is_nil` 来判断是否有效。

```
IndexEntryArray.push(self, IndexEntryRefMut);
IndexEntryArray.push(self, table);
```

```
IndexEntryArray.push
```

在数组末尾附加一个条目。`table` 必须能转为合法的 **IndexEntryRefMut**，表的结构与 `IndexEntryRef.as_table` 返回的一致。

```
IndexEntryArray.extend_from_iterator(self, init);
init := function() →
    function() → IndexEntryRefMut | table | nil;
```

```
IndexEntryArray.extend_from_iterator
```

把迭代器返回的每一项附加到数组中。`init` 返回一个迭代器函数。

```
IndexEntryRefMut() → IndexEntryRefMut;
IndexEntryRefMut(table) → IndexEntryRefMut;
IndexEntryRefMut(IndexEntryRef) → IndexEntryRefMut;
IndexEntryRefMut(IndexEntryRefMut) → IndexEntryRefMut;
```

```
IndexEntryRefMut
```

IndexEntryRefMut 的构造函数。它创建一个新的空索引条目，或从传入的参数中构造出一个新的索引条目。`table` 的结构和 `IndexEntryRef.as_table` 返回的一致。

```
IndexEntryRefMut.push_level(self, key: nil, display: string | StrRef)
    → boolean;
IndexEntryRefMut.push_level(self, level: string | StrRef)
    → boolean;
IndexEntryRefMut.push_level(self, key: string, display: string)
    → boolean;
IndexEntryRefMut.push_level(self, key: StrRef, display: StrRef)
    → boolean;
```

```
IndexEntryRefMut.push_level
```

在条目中插入一个层级。

```
IndexEntryRefMut.set_range(self, range?: StrRef | string) → boolean;
```

```
IndexEntryRefMut.set_range
```

设置本条目是否是页码范围的起始或结束位置。`range` 可选值为 `None`、`Open`、`Close`，如果不设置，则相当于 `None`。返回值表示是否成功设置。

```
IndexEntryRefMut.set_page_commands(
    self,
    page_commands?: StrRef | string
) → boolean;
```

```
IndexEntryRefMut.set_page_commands
```

设置本条目页码要使用的命令。如果未给出或为 `nil`，相当于清除已经设置的命令。

```
IndexEntryRefMut.push_page(
    self,
    kind: StrRef | string,
    value: Into<number>
) → boolean;
```

IndexEntryRefMut.push_page

向本条目中插入一个页码段。页码种类如 `IndexEntryRef.as_table` 所列。

一个条目能同时使用多级页码，如 MMMI-2026-c-8，就依次使用了 Roman, Arabic, alpha, Arabic 四级页码，但只算作一个页码。

```
IndexEntryRefMut.set_pages_raw(self, pages_raw: StrRef | string)
    → boolean;
```

IndexEntryRefMut.set_pages_raw

设置页码的显示效果。在写入到输出文件时，总是使用这个值，页码的具体数值只用于排序等情形，因此它可以不和具体的值挂钩。

```
IndexEntryRefMut.as_table(self) → table;
```

IndexEntryRefMut.as_table

转为 lua 表，它本身的值保持不变。表的结构与 `IndexEntryRef.as_table` 返回的一致。

合并后的索引条目。

MergedEntryRef

```
MergedEntryRef.is_nil(self) → boolean;
MergedEntryRef.to_handle(self) → number;
```

MergedEntryRef._eq
MergedEntryRef.is_nil
MergedEntryRef.to_handle

`MergedEntryRef.is_nil` 用于判断是否为无效值。`MergedEntryRef.to_handle` 用于把条目转为内部句柄。

```
MergedEntryRef.levels_count(self) → number;
MergedEntryRef.level_n(self, n: number) → StrRef | nil;
MergedEntryRef.levels(self)
    → function() → StrRef | nil;
```

MergedEntryRef.levels_count
MergedEntryRef.level_n
MergedEntryRef.levels

返回条目的层级数，或返回对应的层级。

```
MergedEntryRef.key_n(self, n: number) → StrRef | nil;
MergedEntryRef.keys(self) → function() → StrRef | nil;
```

MergedEntryRef.key_n
MergedEntryRef.keys

返回条目的排序关键字。关键字的个数个层级数一致。

```
MergedEntryRef.pages_count(self) → number;
```

MergedEntryRef.pages_count

返回合并后的索引条目包含的页码（或页码区间）数量。

```
MergedEntryRef.page_command_n(self, n: number) → StrRef | false | nil;
MergedEntryRef.page_commands(self) → function() → StrRef | false | nil;
```

MergedEntryRef.page_command_n
MergedEntryRef.page_commands

返回合并后的索引条目的页码在输出时要使用的命令。如果页码没有设置命令，返回 `false`。

```
MergedEntryRef.pages_raw_n(self, n: number)
    → nil | StrRef | (StrRef, StrRef);
MergedEntryRef.pages_raws(self)
    → function() → nil | StrRef | (StrRef, StrRef);
```

MergedEntryRef.pages_raw_n
MergedEntryRef.pages_raw

返回页码要输出的原始字符串。如果是单个页码则返回一个 `StrRef`，如果是页码区间，则返回两个 `StrRef`。

```
MergedEntryRef.page_span_n(self, n: number) → number;
MergedEntryRef.page_spans(self) → function() → number | nil;
```

```
MergedEntryRef.page_span_n
MergedEntryRef.page_spans
```

返回页码区间的长度。如果是无效页码，返回 0，如果是单个页码，返回 1，否则返回区间的长度。

```
MergedEntryRef.max_same_key(MergedEntryRef, MergedEntryRef) → number;
```

```
MergedEntryRef.max_same_key
```

返回两个条目拥有的相同排序键的个数。

```
MergedEntryRef.as_table(self) → table;
table := {
  levels: { { key?: string, display: string }, .. },
  pages?: { page_entry, .. },
}
page_entry := {
  kind: "single",
  page_command: nil | string,
  pages: { { kind: string, value: number }, .. },
  pages_raw: string,
} | {
  kind: "range",
  page_command: nil | string,
  start: { { kind: string, value: number }, .. },
  start_raw: string,
  end: { { kind: string, value: number }, .. },
  end_raw: string,
}
```

```
MergedEntryRef.as_table
```

以 lua 表的形式表示该条目。页码的种类见 [IndexEntryRef.as_table](#)。

3.3 实用类型

```
Logger.error(string | StrRef) → boolean;
```

```
Logger.error
Logger.warn
Logger.info
Logger.debug
Logger.trace
```

在日志中写入对应级别的信息。

```
IstInputStyle: const table;
```

```
IstInputStyle
```

索引的输入格式。

```
IstInputStyle.is_nil(self) → boolean;
```

```
IstInputStyle.is_nil
```

判断是否不存在任何输入格式。一般情况下，输入格式都是存在的。

```
IstInputStyle.keyword() → StrRef | nil;
```

返回输入格式对应关键字的值。

```
IstInputStyle.keyword
IstInputStyle.arg_open
IstInputStyle.arg_close
IstInputStyle.range_open
IstInputStyle.range_close
IstInputStyle.level
IstInputStyle.actual
IstInputStyle.encap
IstInputStyle.quote
IstInputStyle.escape
IstInputStyle.page_compositor
IstInputStyle.comment
IstInputStyle.separator
```

```
IstOutputStyle: const table;
```

IstOutputStyle

索引的输出格式。

```
IstOutputStyle.is_nil(self) → boolean;
```

IstOutputStyle.is_nil

判断是否不存在任何输出格式。一般情况下，输出格式都是存在的。

```
IstOutputStyle.preamble() → StrRef | nil;
```

IstOutputStyle.preamble
IstOutputStyle.postamble
IstOutputStyle.group_skip
IstOutputStyle.heading_prefix
IstOutputStyle.heading_suffix
IstOutputStyle.headings_flag
IstOutputStyle.numhead_positive
IstOutputStyle.numhead_negative
IstOutputStyle.symhead_positive
IstOutputStyle.symhead_negative
IstOutputStyle.item_0
IstOutputStyle.item_1
IstOutputStyle.item_2
IstOutputStyle.item_01
IstOutputStyle.item_x1
IstOutputStyle.item_12
IstOutputStyle.item_x2
IstOutputStyle.delim_0
IstOutputStyle.delim_1
IstOutputStyle.delim_2
IstOutputStyle.delim_n
IstOutputStyle.delim_r
IstOutputStyle.delim_t
IstOutputStyle.encap_prefix
IstOutputStyle.encap_infix
IstOutputStyle.encap_suffix
IstOutputStyle.page_precedence
IstOutputStyle.suffix_2p
IstOutputStyle.suffix_3p
IstOutputStyle.suffix_mp
IstOutputStyle.stroke_prefix
IstOutputStyle.stroke_suffix
IstOutputStyle.radical_prefix
IstOutputStyle.radical_suffix
IstOutputStyle.radical_simplified_flag
IstOutputStyle.radical_simplified_prefix
IstOutputStyle.radical_simplified_separator
IstOutputStyle.radical_simplified_suffix

返回输出格式对应关键字的值。

```
CliOptions: const {  
  c: boolean,  
  q: boolean,  
  r: boolean,  
  strict: boolean,  
  z?: string,  
  extra_options?: { string, .. },  
};
```

CliOptions

命令行选项。extra_options 是命令行参数 -- 之后的内容。

```
CIndex.Util: const table;  
CIndex.Ideographic: const table;  
CIndex.Entry: const table;
```

```
CIndex.Util  
CIndex.Ideographic  
CIndex.Entry
```

cindex 提供的实用模块。

```
CIndex.Util.is_ctype(type, value) → boolean;
```

```
CIndex.Util.is_ctype
```

判断某个值是否为某个 cindex 类型。无法判别所有 lua 原生类型。

例如, `CIndex.Ideographic.is_ctype(StrRef, s)` 即判断 `s` 是否为 `StrRef` 类型。

```
CIndex.Util.table_to_json_string(table) → string;
```

```
CIndex.Util.table_to_json_string
```

把表转为 JSON 字符串。表中只能有 lua 基本类型, 并且有一定限制。

```
local tab = {  
    ["true"] = 114,  
    ["5.14"] = { false },  
};  
print(CIndex.Util.table_to_json_string(tab));  
.....  
{  
  "true": 114,  
  "5.14": [  
    false  
  ]  
}
```

```
CIndex.Util.value_from_json_string(string) → ??;
```

```
CIndex.Util.value_from_json_string
```

把 JSON 字符串转为某个 lua 基本类型。

```
local s = [=[  
  {  
    "true": 114,  
    "5.14": [ false ]  
  }  
]=];  
local res = CIndex.Util.value_from_json_string(s);  
print(  
    type(res),  
    type(res) == "table" and CIndex.Util.table_to_json_string(res)  
);  
.....  
table {  
  "5.14": [  
    false  
  ],  
  "true": 114  
}
```

```
CIndex.Ideographic.group_by_pinyin(Char) → string;
```

```
CIndex.Ideographic.group_by_pinyin  
CIndex.Ideographic.group_by_bihua  
CIndex.Ideographic.group_by_bushou
```

返回字符按拼音/笔画数/部首的组别，组别见 `cindex.table.GroupOrder`。

如果字符不位于 CJK 表意字相关区块或没有对应信息，则返回 `Symbols`，即认为它是符号组。

除了 CJK 表意字相关区块的字符外，还识别表 6 补充的字符的信息。

字符	代码点	拼音	部首	笔顺
〇	U+3007	líng	乙 (5)	5

表 6: 补充的字符信息

```
CIndex.Entry.compare_by_default(MergedEntryRef, MergedEntryRef)  
→ boolean;
```

```
CIndex.Entry.compare_by_default  
CIndex.Entry.compare_by_pinyin  
CIndex.Entry.compare_by_bihua  
CIndex.Entry.compare_by_bushou
```

根据指定的比较算法比较两个条目，如果前者应当排在后者之前，则返回 `true`。

总体来说，`cindex` 首先进行忽略大小写的比较，若相等，再区分大小写进行比较，顺序按照 Unicode 的顺序。若汉字与非汉字进行比较，总是非汉字排在前面，而不考虑它们在 Unicode 中的顺序。

使用读音排序时，没有读音数据的字符（包括生僻字）一律排在有读音的字符之前，汉字按其最常用读音比较，读音相同汉字的按 Unicode 编码排序。

使用笔画数和笔顺排序时，笔画数小的排在前面，笔画数相同的，按横、竖、撇、点（捺）、折的顺序逐笔画比较，仍然相同的按 Unicode 编码排序；生僻汉字没有笔顺信息的，排在同笔画数有笔顺的字后面。

使用部首和除部首笔画数排序时，部首按康熙字典 214 部首顺序排列，部首和笔画数相同的按 Unicode 编码排序。

3.4 PCRE2 正则表达式

PCRE2 是 C 语言实现正则表达式库。关于它的相关信息和语法，见 <https://pcre2project.github.io/pcre2/doc/>。

`cindex` 对其做了简单封装。

```
Regex(string | StrRef | const uint8_t*, options: number) → Regex;
```

```
Regex
```

创建一个正则表达式。无效的模式直接报错。

```
Regex.jit_compile(self) → true | (false, string);
```

```
Regex.jit_compile
```

尝试使用 JIT 编译。返回是否成功。如果为 `false`，还返回相关错误码。

```
Regex.is_match(self, subject: StrRef | string) → boolean;
```

```
Regex.is_match
```

检查 `subject` 是否匹配正则表达式。

```
local re = Regex([==[\w?\d+\p{Script=Han}]==]);  
print(re.is_match("我114514谁? "));  
print(re.is_match("T114514谁? "));  
print(re.is_match("T114514L"));
```

```

.....
true
true
false

```

```

Regex.find(self, subject: StrRef, start_pos: number) → StrRef | nil;
Regex.find(self, subject: string, start_pos: number) → string | nil;

```

Regex.find

从字符串的 `start_pos` 位置开始，查找匹配该的正则表达式的子串并返回。不论是 `StrRef` 还是 `string`，索引均从 0 开始。

```

-- 可选匹配“字母”，然后是多个数字，然后是具有属性 Script=Han 的字符
local re = Regex([==[\pL?\d+\p{Script=Han}]==]);
print(re:find("999我辈456子"));
print(re:find("999我辈456子", 3));
print(re:find("我456"));
.....
999我
  辈456子
nil

```

```

Regex.captures(self, subject: StrRef, start_pos: number)
    → nil | { [number] = StrRef, [string] = StrRef, .. };
Regex.captures(self, subject: string, start_pos: number)
    → nil | { [number] = string, [string] = string, .. };

```

Regex.captures

查找并返回字符串的捕获组。第 0 个索引是匹配的子串。

```

local function print_value(value)
    if type(value) == "table" then
        print("{");
        for k, v in pairs(value) do
            print(" ", k, "=", v);
        end
        print("}");
    else
        print(value);
    end
end

local re = Regex([==[(\pL)?\d+(\p{Script=Han})]==]);
print_value(re:captures("999我辈456子"));
print_value(re:captures("999我辈456子", 3));
print_value(re:captures("我456"));
.....
{
  0 = 999我
  1 = false
  2 = 我
}
{
  0 = 辈456子
}

```

```

1 = 辈
2 = 子
}
nil

```

```

Regex.gmatch(self, subject: StrRef, start_pos: number)
  → function() → nil | { [number] = StrRef, [string] = StrRef, .. };
Regex.gmatch(self, subject: string, start_pos: number)
  → function() → nil | { [number] = string, [string] = string, .. };

```

Regex.gmatch

返回一个迭代器函数，其返回值依次是匹配的捕获。

```

Regex.gsub(
  self,
  subject: StrRef | string,
  replacement: StrRef | string,
  options?: number,
  once?: boolean;
) → string, number;

```

Regex.gsub

以正则表达式 `replacement` 替换 `subject` 中匹配的内容。返回替换后的字符串以及替换次数。

```

local bit = require("bit");
local re = Regex([==(\\pL)?\\d+(\\p{Script=Han})==]);
print(re.gsub("999我辈456子", "$2,$1"));
print(re.gsub("999我辈456子", "$2,$1", nil, true));
.....
[我,][子,辈] 2
[我,]辈456子 1

```

```

Regex.Constant: const { [string] = number, .. };

```

Regex.Constant

这个表保存一些 PCRE2 定义的常量。

```

Regex.Constant.UNSET: number;

```

Regex.Constant.UNSET
Regex.Constant.ZERO_TERMINATED

`Regex.Constant.ALLOW_EMPTY_CLASS: number;`

`Regex.Constant.ALLOW_EMPTY_CLASS`
`Regex.Constant.ALT_BSUX`
`Regex.Constant.AUTO_CALLOUT`
`Regex.Constant.CASELESS`
`Regex.Constant.DOLLAR_ENDONLY`
`Regex.Constant.DOTALL`
`Regex.Constant.DUPNAMES`
`Regex.Constant.EXTENDED`
`Regex.Constant.FIRSTLINE`
`Regex.Constant.MATCH_UNSET_BACKREF`
`Regex.Constant.MULTILINE`
`Regex.Constant.NEVER_UCP`
`Regex.Constant.NEVER_UTF`
`Regex.Constant.NO_AUTO_CAPTURE`
`Regex.Constant.NO_AUTO_POSSESS`
`Regex.Constant.NO_DOTSTAR_ANCHOR`
`Regex.Constant.NO_START_OPTIMIZE`
`Regex.Constant.UCP`
`Regex.Constant.UNGREEDY`
`Regex.Constant.UTF`
`Regex.Constant.NEVER_BACKSLASH_C`
`Regex.Constant.ALT_CIRCUMFLEX`
`Regex.Constant.ALT_VERBNAMES`
`Regex.Constant.USE_OFFSET_LIMIT`
`Regex.Constant.EXTENDED_MORE`
`Regex.Constant.LITERAL`
`Regex.Constant.MATCH_INVALID_UTF`
`Regex.Constant.ALT_EXTENDED_CLASS`

`Regex.Constant.NOTBOL: number;`

`Regex.Constant.NOTBOL`
`Regex.Constant.NOTEOL`
`Regex.Constant.NOTEMPTY`
`Regex.Constant.NOTEMPTY_ATSTART`
`Regex.Constant.PARTIAL_SOFT`
`Regex.Constant.PARTIAL_HARD`
`Regex.Constant.DFA_RESTART`
`Regex.Constant.DFA_SHORTEST`
`Regex.Constant.SUBSTITUTE_GLOBAL`
`Regex.Constant.SUBSTITUTE_EXTENDED`
`Regex.Constant.SUBSTITUTE_UNSET_EMPTY`
`Regex.Constant.SUBSTITUTE_UNKNOWN_UNSET`
`Regex.Constant.SUBSTITUTE_OVERFLOW_LENGTH`
`Regex.Constant.NO_JIT`
`Regex.Constant.COPY_MATCHED_SUBJECT`
`Regex.Constant.SUBSTITUTE_LITERAL`
`Regex.Constant.SUBSTITUTE_MATCHED`
`Regex.Constant.SUBSTITUTE_REPLACEMENT_ONLY`

```
Regex.Constant.INFO_ALLOPTIONS: number;
```

```
Regex.Constant.INFO_ALLOPTIONS
Regex.Constant.INFO_ARGOPTIONS
Regex.Constant.INFO_BACKREFMAX
Regex.Constant.INFO_BSR
Regex.Constant.INFO_CAPTURECOUNT
Regex.Constant.INFO_FIRSTCODEUNIT
Regex.Constant.INFO_FIRSTCODETYPE
Regex.Constant.INFO_FIRSTBITMAP
Regex.Constant.INFO_HASCORRLF
Regex.Constant.INFO_JCHANGED
Regex.Constant.INFO_JITSIZE
Regex.Constant.INFO_LASTCODEUNIT
Regex.Constant.INFO_LASTCODETYPE
Regex.Constant.INFO_MATCHEMPTY
Regex.Constant.INFO_MATCHLIMIT
Regex.Constant.INFO_MAXLOOKBEHIND
Regex.Constant.INFO_MINLENGTH
Regex.Constant.INFO_NAMECOUNT
Regex.Constant.INFO_NAMEENTRYSIZE
Regex.Constant.INFO_NAMETABLE
Regex.Constant.INFO_NEWLINE
Regex.Constant.INFO_DEPTHLIMIT
Regex.Constant.INFO_RECURSIONLIMIT
Regex.Constant.INFO_SIZE
Regex.Constant.INFO_HASBACKSLASHC
Regex.Constant.INFO_FRAME_SIZE
Regex.Constant.INFO_HEAPLIMIT
Regex.Constant.INFO_EXTRAOPTIONS
```

§ 4 可加载的外部库

cindex 对加载外部 lua 文件和库做了限制，目前只允许加载某些特定的库。

4.1 cindex.table

cindex 的相关表。可使用 `require("cindex.table")` 加载。

cindex.table

```
cindex.table.GroupOrder: const {
  "nil" = 0,
  "Symbols" = ??,
  "Numbers" = ??,
  "A" = ??, .., "Z" = ??,
  "BiHua1" = ??, .., "BiHua64" = ??,
  "BuShou1" = ??, .., "BuShou214" = ??,
  [number] = string, ..
};
```

cindex.table.GroupOrder

cindex 默认的各个组别及其排序。Symbols 是符号分组，Numbers 是数字分组，BiHua.. 是笔画数分组，BuShou.. 是康熙部首分组。每个分组对应一个数字。也可通过数字反查对应的组别名。

```
BiHuaToGroup: const { [1] = string, .., [64] = string };
```

cindex.table.BiHuaToGroup

根据笔画数返回组别名。

```
BuShouToGroup: const { [number] = string, .. }
```

cindex.table.BuShouToGroup

根据康熙部首的代码点（包括康熙部首相关区块和表意字相关区块）值返回组别名。

```
BuShouToChars: const {
  [1] = { number, .. }, .., [214] = { number, .. },
  ["BuShou1"] = { number, .. }, .., ["BuShou214"] = { number, .. },
};
```

`cindex.table.BuShouToChars`

康熙部首对应的表意字字符，如果有变体，则依次存储在对应的表中。

4.2 icu.table

`icu.table` 保存 Unicode 相关的信息。可使用 `require("icu.table")` 加载。

`icu.table`

```
Script = { [number] = string, [string] = number, .. }
```

`icu.table.Script`

`Char.script` 返回的句柄可用于索引此表。也可通过字符串反查对应的句柄。

```
GeneralCategory = { [number] = string, [string] = number, .. }
```

`icu.table.GeneralCategory`

`Char.general_category` 返回的句柄可用于索引此表。也可通过字符串反查对应的句柄。

```
EmojiGroup = { [number] = group, [string] = number, .. }
group := |
  "Activities"
  "Animals And Nature"
  "Flags"
  "Food And Drink"
  "Objects"
  "People And Body"
  "Smileys And Emotion"
  "Symbols"
  "Travel And Places"
  ??
```

`icu.table.EmojiGroup`

Emoji 的分组。目前共有 9 个组别。

```
PropertyShortNames = { [string] = string, .. }
```

`icu.table.PropertyShortNames`

属性值对应的短名。

4.3 digest 哈希函数

`cindex` 封装了 MD5, Sha2 以及 Blake2 哈希函数。可使用 `require("digest")` 加载。

```
digest: const table;
```

`digest`

```
digest.md5(string) → string;
```

`digest.md5`
`digest.sha256`
`digest.sha512`
`digest.blake2b128`
`digest.blake2b256`

```
local digest = require("digest");
print(digest.md5("Hello, world!"));
print(digest.sha256("Hello, world!"));
print(digest.blake2b256("Hello, world!"));
.....
```

```
6cd3556deb0da54bca060b4c39479839
315f5bdb76d078c43b8ac0064e4a0164612b1fceb77c869345bfc94c75894edd3
b5da441cfe72ae042ef4d2b17742907f675de4da57462d4c3609c2e2ed755970
```

4.4 LPeg

LPeg 是基于 Parsing Expression Grammars 的模式匹配库。cindex 内置了此库的 1.1.0 版，可使用 `require("lpeg")` 加载。详细用法见 <https://www.inf.puc-rio.br/~roberto/lpeg/lpeg.html>。

4.5 argparse

argparse 是解析命令行的 lua 库。cindex 内置了此库的 0.7.2 版，可使用 `require("argparse")` 加载。详细用法见 <http://argparse.readthedocs.org/>。

cindex 支持在 lua 格式中解析命令行传入的额外的参数，可使用此库解析。

```
-- 假定额外的参数如下
local extra_options = { "-o", "cindexout.tex", "cindexin.tex" };
local argparse = require("argparse");
local parser = argparse("mystyle");
parser:argument("input", "Input file.");
parser:option("-o --output", "Output file.", "a.out");
parser:option("-I --include", "Include locations."):count("*");
local args = parser:parse(extra_options);
print(CIndex.Util.table_to_json_string(args));

.....
{
  "include": {},
  "input": "cindexin.tex",
  "output": "cindexout.tex"
}
```

§ 5 代码索引

粗体的数字表示描述对应索引项的页码；意大利体的数字表示使用对应索引项的页码。

G

格式

概念

数字	3, 4, 5
注释	3
转义符	3
字符	3, 3, 4
字符串	3, 3-5

关键字

actual	4
arg_close	4
arg_open	3
comment	4
delim_0	5
delim_1	5
delim_2	5
delim_n	5
delim_r	5
delim_t	5
encap	4
encap_infix	5
encap_prefix	5
encap_suffix	5
escape	4
group_skip	4
heading_prefix	4
heading_suffix	4
headings_flag	4
indent_length	5
indent_space	5, 5
item_0	4
item_01	4
item_1	4
item_12	4
item_2	4
item_x1	4
item_x2	4
keyword	3
level	4
line_max	5
numhead_negative	4
numhead_positive	4
page_compositor	4
page_precedence	5
postamble	4

preamble	4
quote	4
radical_prefix	5
radical_simplified_flag	5
radical_simplified_prefix	5
radical_simplified_separator	5
radical_simplified_suffix	5
radical_suffix	5
range_close	4
range_open	4
separator	4
setpage_prefix	5
setpage_suffix	5
stroke_prefix	5
stroke_suffix	5
suffix_2p	5
suffix_3p	5
suffix_mp	5
symhead_negative	4
symhead_positive	4

L

lua 类型

__ipairs	16
Char	7, 7, 8
__eq	7
__le	7
__lt	7
__tostring	7
chars	8
cmp	8
from_number	8
from_string	8
general_category	9, 28
han_info	9, 13
han_ordered_strokes	9
has_han_info	9, 14
is_alphabetic	9
is_ideographic	9
is_math	9
is_numeric	9
is_valid	8
script	9, 28
simple_fold	10, 10

to_number	8	group	16
CIndex		name	16
Entry	22	empty_indices_callout	6
Ideographic	22	entry_compare	7
Util	22	group_compare	7
CIndex.Entry		group_detect	6
compare_by_bihua	23	icu.table	28, 28
compare_by_bushou	23	EmojiGroup	16, 28
compare_by_default	23	GeneralCategory	9, 28
compare_by_pinyin	23	PropertyShortNames	28
CIndex.Ideographic		Script	9, 28
group_by_bihua	23	IndexEntryArray	17, 17
group_by_bushou	23	__index	18
group_by_pinyin	23	__ipairs	17
cindex.table	27	__len	17
BiHuaToGroup	27	extend_from_iterator	18
BuShouToChars	28	get	18
BuShouToGroup	27	is_empty	17
GroupOrder	23, 27	is_nil	17
CIndex.Util		push	18
is_ctype	22	IndexEntryRef	17, 17, 18
table_to_json_string	22	__eq	17
value_from_json_string	22	as_table	6, 17, 18–20
CjkInfo	13, 14	is_nil	17, 18
__eq	13	is_same	17
__tostring	13	key_n	17
cantonese	15	keys	17
contains_char	14	levels_count	17
is_nil	14	to_handle	17
kangxi_radical_ideography_slot	15	IndexEntryRefMut	6, 18, 18
kangxi_radical_number	14	as_table	19
kx_ideography_char	14	push_level	18
kx_radical_char	14	push_page	19
kx_radical_kind	15	set_page_commands	18
mandarin	15	set_pages_raw	19
original_radical	15	set_range	18
radical_from_kind	14	IstInputStyle	20
total_strokes	15, 15	actual	20
ucd_additional_strokes	15	arg_close	20
ucd_total_strokes	15, 15	arg_open	20
CliOptions	21	comment	20
digest	28	encap	20
blake2b128	28	escape	20
blake2b256	28	is_nil	20
md5	28	keyword	20
sha256	28	level	20
sha512	28	page_compositor	20
Emoji	16	quote	20
__tostring	16	range_close	20

range_open	20	__eq	19
separator	20	as_table	20
IstOutputStyle	21	is_nil.....	19, 19
delim_0.....	21	key_n	19
delim_1.....	21	keys	19
delim_2.....	21	level_n.....	19
delim_n.....	21	levels.....	19
delim_r.....	21	levels_count	19
delim_t.....	21	max_same_key	20
encap_infix	21	page_command_n.....	19
encap_prefix	21	page_commands	19
encap_suffix	21	page_span_n	20
group_skip	21	page_spans	20
heading_prefix.....	21	pages_count	19
heading_suffix.....	21	pages_raw	19
headings_flag	21	pages_raw_n	19
is_nil.....	21	to_handle	19, 19
item_0.....	21	OrderedStrokes	16, 16
item_01.....	21	__eq	16
item_1.....	21	__gt	16
item_12.....	21	__index.....	16
item_2.....	21	__ipairs.....	16
item_x1.....	21	__len	16
item_x2.....	21	__lt	16
numhead_negative.....	21	__tostring	16
numhead_positive.....	21	get	16
page_precedence.....	21	output_style	7
postamble	21	page_precedence	6
preamble.....	21	parse_index_line	6
radical_prefix.....	21	parse_indices_string.....	6, 6
radical_simplified_flag.....	21	Regex	23
radical_simplified_prefix	21	captures	24
radical_simplified_separator	21	Constant.....	25
radical_simplified_suffix	21	find	24
radical_suffix.....	21	gmatch.....	25
stroke_prefix	21	gsub	25
stroke_suffix	21	is_match.....	23
suffix_2p	21	jit_compile	23
suffix_3p	21	Regex.Constant	
suffix_mp	21	ALLOW_EMPTY_CLASS	26
symhead_negative.....	21	ALT_BSUX	26
symhead_positive.....	21	ALT_CIRCUMFLEX.....	26
Logger		ALT_EXTENDED_CLASS	26
debug	20	ALT_VERBNAMES	26
error	20	AUTO_CALLOUT	26
info	20	CASELESS	26
trace	20	COPY_MATCHED_SUBJECT	26
warn	20	DFA_RESTART	26
MergedEntryRef	19	DFA_SHORTEST	26

DOLLAR_ENDONLY	26	NO_START_OPTIMIZE	26
DOTALL	26	NOTBOL	26
DUPNAMES	26	NOTEMPTY	26
EXTENDED	26	NOTEMPTY_ATSTART	26
EXTENDED_MORE	26	NOTEOL	26
FIRSTLINE	26	PARTIAL_HARD	26
INFO_ALLOPTIONS	27	PARTIAL_SOFT	26
INFO_ARGOPTIONS	27	SUBSTITUTE_EXTENDED	26
INFO_BACKREFMAX	27	SUBSTITUTE_GLOBAL	26
INFO_BSR	27	SUBSTITUTE_LITERAL	26
INFO_CAPTURECOUNT	27	SUBSTITUTE_MATCHED	26
INFO_DEPTHLIMIT	27	SUBSTITUTE_OVERFLOW_LENGTH	26
INFO_EXTRAOPTIONS	27	SUBSTITUTE_REPLACEMENT_ONLY	26
INFO_FIRSTBITMAP	27	SUBSTITUTE_UNKNOWN_UNSET	26
INFO_FIRSTCODETYPE	27	SUBSTITUTE_UNSET_EMPTY	26
INFO_FIRSTCODEUNIT	27	UCP	26
INFO_FRAME_SIZE	27	UNGREEDY	26
INFO_HASBACKSLASHC	27	UNSET	25
INFO_HASCORRLF	27	USE_OFFSET_LIMIT	26
INFO_HEAPLIMIT	27	UTF	26
INFO_JCHANGED	27	ZERO_TERMINATED	25
INFO_JITSIZE	27	StrRef	10, 10, 19, 22, 24
INFO_LASTCODETYPE	27	__eq	10
INFO_LASTCODEUNIT	27	__le	10
INFO_MATCHEMPTY	27	__len	10
INFO_MATCHLIMIT	27	__lt	10
INFO_MAXLOOKBEHIND	27	__tostring	10
INFO_MINLENGTH	27	chars	11
INFO_NAMECOUNT	27	cmp	10
INFO_NAMEENTRYSIZE	27	cmp_ignore_case	10
INFO_NAMETABLE	27	eq_ignore_case	10, 10
INFO_NEWLINE	27	from_ptr	10
INFO_RECURSIONLIMIT	27	graphemes	12
INFO_SIZE	27	head	11
LITERAL	26	is_emoji	13
MATCH_INVALID_UTF	26	is_empty	10, 10
MATCH_UNSET_BACKREF	26	is_nil	10, 10
MULTILINE	26	lines	11
NEVER_BACKSLASH_C	26	slice	11
NEVER_UCP	26	split	11
NEVER_UTF	26	split_word_bounds	12
NO_AUTO_CAPTURE	26	to_string_escaped	10
NO_AUTO_POSSESS	26	trim_ascii_spaces	13
NO_DOTSTAR_ANCHOR	26	unicode_words	12
NO_JIT	26	unsafe_new	10